

Still, there's no clear winner between the object-oriented and more data-oriented versions of this code. They both do the same thing. If we have new functions that accept a polygon argument, such as `area(polygon)` or `point_in_polygon(polygon, x, y)`, the benefits of the object-oriented code become increasingly obvious. Likewise, if we add other attributes to the polygon, such as `color` or `texture`, it makes more and more sense to encapsulate that data into a single class.

The distinction is a design decision, but in general, the more important a set of data is, the more likely it is to have multiple functions specific to that data, and the more useful it is to use a class with attributes and methods instead.

When making this decision, it also pays to consider how the class will be used. If we're only trying to calculate the perimeter of one polygon in the context of a much greater problem, using a function will probably be quickest to code and easier to use *one time only*. On the other hand, if our program needs to manipulate numerous polygons in a wide variety of ways (calculating the perimeter, area, and intersection with other polygons, moving or scaling them, and so on), we have almost certainly identified an object; one that needs to be extremely versatile.

Additionally, pay attention to the interaction between objects. Look for inheritance relationships; inheritance is impossible to model elegantly without classes, so make sure to use them. Look for the other types of relationships we discussed in Chapter 1, *Object-Oriented Design*, association and composition. Composition can, technically, be modeled using only data structures; for example, we can have a list of dictionaries holding tuple values, but it is sometimes less complicated to create a few classes of objects, especially if there is behavior associated with the data.



Don't rush to use an object just because you can use an object, but don't neglect to create a class when you need to use a class.

Adding behaviors to class data with properties

Throughout this book, we've focused on the separation of behavior and data. This is very important in object-oriented programming, but we're about to see that, in Python, the distinction is uncannily blurry. Python is very good at blurring distinctions; it doesn't exactly help us to *think outside the box*. Rather, it teaches us to stop thinking about the box.

Before we get into the details, let's discuss some bad object-oriented theory. Many object-oriented languages teach us to never access attributes directly (Java is the most notorious). They insist that we write attribute access like this:

```
class Color:
    def __init__(self, rgb_value, name):
        self._rgb_value = rgb_value
        self._name = name

    def set_name(self, name):
        self._name = name

    def get_name(self):
        return self._name
```

The variables are prefixed with an underscore to suggest that they are private (other languages would actually force them to be private). Then, the `get` and `set` methods provide access to each variable. This class would be used in practice as follows:

```
>>> c = Color("#ff0000", "bright red")
>>> c.get_name()
'bright red'
>>> c.set_name("red")
>>> c.get_name()
'red'
```

This is not nearly as readable as the direct access version that Python favors:

```
class Color:
    def __init__(self, rgb_value, name):
        self.rgb_value = rgb_value
        self.name = name

c = Color("#ff0000", "bright red")
print(c.name)
c.name = "red"
print(c.name)
```

So, why would anyone insist upon the method-based syntax? Their reasoning is that, someday, we may want to add extra code when a value is set or retrieved. For example, we could decide to cache a value to avoid complex computations, or we might want to validate that a given value is a suitable input.

In code, for example, we could decide to change the `set_name()` method as follows:

```
def set_name(self, name):
    if not name:
        raise Exception("Invalid Name")
    self._name = name
```

Now, in Java and similar languages, if we had written our original code for direct attribute access, and then later changed it to a method like the preceding one, we'd have a problem: anyone who had written code that accessed the attribute directly would now have to access a method. If they didn't then change the access style from attribute access to a function call, their code will be broken.

The mantra in these languages is that we should never make public members private. This doesn't make much sense in Python since there isn't any real concept of private members!

Python gives us the `property` keyword to make methods that *look* like attributes. We can therefore write our code to use direct member access, and if we ever unexpectedly need to alter the implementation to do some calculation when getting or setting that attribute's value, we can do so without changing the interface. Let's see how it looks:

```
class Color:
    def __init__(self, rgb_value, name):
        self.rgb_value = rgb_value
        self._name = name

    def _set_name(self, name):
        if not name:
            raise Exception("Invalid Name")
        self._name = name

    def _get_name(self):
        return self._name

    name = property(_get_name, _set_name)
```

Compared to the earlier class, we first change the `name` attribute into a (semi-)private `_name` attribute. Then, we add two more (semi-)private methods to get and set that variable, performing our validation when we set it.

Finally, we have the `property` declaration at the bottom. This is the Python magic. It creates a new attribute on the `Color` class called `name`, to replace the direct `name` attribute. It sets this attribute to be a **property**. Under the hood, `property` calls the two methods we just created whenever the value is accessed or changed. This new version of the `Color` class can be used exactly the same way as the earlier version, yet it now performs validation when we set the `name` attribute:

```
>>> c = Color("#0000ff", "bright red")
>>> print(c.name)
bright red
>>> c.name = "red"
>>> print(c.name)
red
>>> c.name = ""
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "setting_name_property.py", line 8, in _set_name
    raise Exception("Invalid Name")
Exception: Invalid Name
```

So, if we'd previously written code to access the `name` attribute, and then changed it to use our `property`-based object, the previous code would still work, unless it was sending an empty `property` value, which is the behavior we wanted to forbid in the first place. Success!

Bear in mind that, even with the `name` `property`, the previous code is not 100% safe. People can still access the `_name` attribute directly and set it to an empty string if they want to. But if they access a variable we've explicitly marked with an underscore to suggest it is private, they're the ones that have to deal with the consequences, not us.

Properties in detail

Think of the `property` function as returning an object that proxies any requests to set or access the attribute value through the methods we have specified. The `property` built-in is like a constructor for such an object, and that object is set as the public-facing member for the given attribute.

This property constructor can actually accept two additional arguments, a `delete` function and a docstring for the property. The `delete` function is rarely supplied in practice, but it can be useful for logging the fact that a value has been deleted, or possibly to veto deleting if we have reason to do so. The docstring is just a string describing what the property does, no different from the docstrings we discussed in *Chapter 2, Objects in Python*. If we do not supply this parameter, the docstring will instead be copied from the docstring for the first argument: the `getter` method. Here is a silly example that states whenever any of the methods are called:

```
class Silly:
    def _get_silly(self):
        print("You are getting silly")
        return self._silly

    def _set_silly(self, value):
        print("You are making silly {}".format(value))
        self._silly = value

    def _del_silly(self):
        print("Whoah, you killed silly!")
        del self._silly

    silly = property(_get_silly, _set_silly, _del_silly, "This is a silly
property")
```

If we actually use this class, it does indeed print out the correct strings when we ask it to:

```
>>> s = Silly()
>>> s.silly = "funny"
You are making silly funny
>>> s.silly
You are getting silly
'funny'
>>> del s.silly
Whoah, you killed silly!
```

Further, if we look at the help file for the `Silly` class (by issuing `help(Silly)` at the interpreter prompt), it shows us the custom docstring for our `silly` attribute:

```
Help on class Silly in module __main__:

class Silly(builtins.object)
|   Data descriptors defined here:
|
|   __dict__
|       dictionary for instance variables (if defined)
|
```

```
| __weakref__  
|     list of weak references to the object (if defined)  
|  
| silly  
|     This is a silly property
```

Once again, everything is working as we planned. In practice, properties are normally only defined with the first two parameters: the `getter` and `setter` functions. If we want to supply a docstring for a property, we can define it on the `getter` function; the property proxy will copy it into its own docstring. The `delete` function is often left empty because object attributes are so rarely deleted. If a coder does try to delete a property that doesn't have a `delete` function specified, it will raise an exception. Therefore, if there is a legitimate reason to delete our property, we should supply that function.

Decorators – another way to create properties

If you've never used Python decorators before, you might want to skip this section and come back to it after we've discussed the decorator pattern in Chapter 10, *Python Design Patterns I*. However, you don't need to understand what's going on to use the decorator syntax in order to make property methods more readable.

The `property` function can be used with the decorator syntax to turn a `get` function into a property function, as follows:

```
class Foo:  
    @property  
    def foo(self):  
        return "bar"
```

This applies the `property` function as a decorator, and is equivalent to the previous `foo = property(foo)` syntax. The main difference, from a readability perspective, is that we get to mark the `foo` function as a property at the top of the method, instead of after it is defined, where it can be easily overlooked. It also means we don't have to create private methods with underscore prefixes just to define a property.

Going one step further, we can specify a `setter` function for the new property as follows:

```
class Foo:  
    @property  
    def foo(self):  
        return self._foo  
  
    @foo.setter  
    def foo(self, value):
```

```
self._foo = value
```

This syntax looks pretty odd, although the intent is obvious. First, we decorate the `foo` method as a getter. Then, we decorate a second method with exactly the same name by applying the `setter` attribute of the originally decorated `foo` method! The `property` function returns an object; this object always comes with its own `setter` attribute, which can then be applied as a decorator to other functions. Using the same name for the get and set methods is not required, but it does help to group together the multiple methods that access one property.

We can also specify a `delete` function with `@foo.deleter`. We cannot specify a docstring using `property` decorators, so we need to rely on the property copying the docstring from the initial getter method. Here's our previous `Silly` class rewritten to use `property` as a decorator:

```
class Silly:
    @property
    def silly(self):
        "This is a silly property"
        print("You are getting silly")
        return self._silly

    @silly.setter
    def silly(self, value):
        print("You are making silly {}".format(value))
        self._silly = value

    @silly.deleter
    def silly(self):
        print("Whoah, you killed silly!")
        del self._silly
```

This class operates *exactly* the same as our earlier version, including the help text. You can use whichever syntax you feel is more readable and elegant.

Deciding when to use properties

With the built-in property clouding the division between behavior and data, it can be confusing to know when to choose an attribute, or a method, or a property. The use case example we saw earlier is one of the most common uses of properties; we have some data on a class that we later want to add behavior to. There are also other factors to take into account when deciding to use a property.

Technically, in Python, data, properties, and methods are all attributes on a class. The fact that a method is callable does not distinguish it from other types of attributes; indeed, we'll see in Chapter 7, *Python Object-Oriented Shortcuts*, that it is possible to create normal objects that can be called like functions. We'll also discover that functions and methods are themselves normal objects.

The fact that methods are just callable attributes, and properties are just customizable attributes, can help us make this decision. Methods should typically represent actions; things that can be done to, or performed by, the object. When you call a method, even with only one argument, it should *do* something. Method names are generally verbs.

Once confirming that an attribute is not an action, we need to decide between standard data attributes and properties. In general, always use a standard attribute until you need to control access to that property in some way. In either case, your attribute is usually a noun. The only difference between an attribute and a property is that we can invoke custom actions automatically when a property is retrieved, set, or deleted.

Let's look at a more realistic example. A common need for custom behavior is caching a value that is difficult to calculate or expensive to look up (requiring, for example, a network request or database query). The goal is to store the value locally to avoid repeated calls to the expensive calculation.

We can do this with a custom getter on the property. The first time the value is retrieved, we perform the lookup or calculation. Then, we can locally cache the value as a private attribute on our object (or in dedicated caching software), and the next time the value is requested, we return the stored data. Here's how we might cache a web page:

```
from urllib.request import urlopen

class WebPage:
    def __init__(self, url):
        self.url = url
        self._content = None

    @property
    def content(self):
        if not self._content:
            print("Retrieving New Page...")
            self._content = urlopen(self.url).read()
        return self._content
```

We can test this code to see that the page is only retrieved once:

```
>>> import time
>>> webpage = WebPage("http://ccphillips.net/")
>>> now = time.time()
>>> content1 = webpage.content
Retrieving New Page...
>>> time.time() - now
22.43316888809204
>>> now = time.time()
>>> content2 = webpage.content
>>> time.time() - now
1.9266459941864014
>>> content2 == content1
True
```

I was on an awful satellite connection when I originally tested this code for the first version of this book back in 2010 and it took 20 seconds the first time I loaded the content. The second time, I got the result in 2 seconds (which is really just the amount of time it took to type the lines into the interpreter). On my more modern connection it looks as follows:

```
>>> webpage = WebPage("https://dusty.phillips.codes")
>>> import time
>>> now = time.time() ; content1 = webpage.content ; print(time.time() -
now)
Retrieving New Page...
0.6236202716827393
>>> now = time.time() ; content2 = webpage.content ; print(time.time() -
now)
1.7881393432617188e-05M
```

It takes about 620 milliseconds to retrieve a page from my web host. From my laptop's RAM, it takes 0.018 milliseconds!

Custom getters are also useful for attributes that need to be calculated on the fly, based on other object attributes. For example, we might want to calculate the average for a list of integers:

```
class AverageList(list):
    @property
    def average(self):
        return sum(self) / len(self)
```

This very simple class inherits from `list`, so we get list-like behavior for free. We just add a property to the class, and hey presto, our list can have an average as follows:

```
>>> a = AverageList([1, 2, 3, 4])
>>> a.average
2.5
```

Of course, we could have made this a method instead, but then we ought to call it `calculate_average()`, since methods represent actions. But a property called `average` is more suitable, and is both easier to type and easier to read.

Custom setters are useful for validation, as we've already seen, but they can also be used to proxy a value to another location. For example, we could add a content setter to the `WebPage` class that automatically logs into our web server and uploads a new page whenever the value is set.

Manager objects

We've been focused on objects and their attributes and methods. Now, we'll take a look at designing higher-level objects; the kind of objects that manage other objects – the objects that tie everything together.

The difference between these objects and most of the previous examples is that the latter usually represent concrete ideas. Management objects are more like office managers; they don't do the actual *visible* work out on the floor, but without them, there would be no communication between departments and nobody would know what they are supposed to do (although, this can be true anyway if the organization is badly managed!). Analogously, the attributes on a management class tend to refer to other objects that do the *visible* work; the behaviors on such a class delegate to those other classes at the right time, and pass messages between them.

As an example, we'll write a program that does a find-and-replace action for text files stored in a compressed ZIP file. We'll need objects to represent the ZIP file and each individual text file (luckily, we don't have to write these classes, as they're available in the Python standard library). The manager object will be responsible for ensuring the following three steps occur in order:

1. Unzipping the compressed file
2. Performing the find-and-replace action
3. Zipping up the new files